Folio: A NotebookLM-Style Retrieval-Augmented Generation System

1 Oct 6, 2026 · @swayam

Abstract

Folio is a multi-user web application that answers questions about a user's own PDF documents and cites the exact page behind every claim. It applies retrieval-augmented generation (RAG): documents are split into page-tagged chunks, embedded locally with `BAAI/bge-small-en-v1.5` (384 dimensions), and stored in PostgreSQL with the pgvector extension. At question time the six most similar chunks are retrieved and passed to a hosted large language model (`openai/gpt-oss-120b` on Groq's free tier) with instructions to cite each claim using numbered markers.

The answer streams to a React interface, where markers become clickable citation chips that open the source PDF at the cited page with the supporting passage highlighted. The system adds hybrid vector and keyword retrieval, OCR for scanned pages, generated quizzes and flashcards whose every item cites a page, username and password authentication, per-user data isolation, background document processing, persistent chat history and a one-command Docker Compose deployment. All components run at zero cost: embeddings are computed on the host CPU, and only answer generation calls an external API. End-to-end browser testing confirmed the full workflow, failure handling and data persistence across restarts.

1. Introduction

1.1 Problem statement

General-purpose language models answer fluently but cannot see a user's private documents, and when they lack information they may invent plausible text (hallucination). Students, researchers and professionals who need answers from specific reports, papers or notes therefore cannot rely on a chatbot alone. Even a correct answer is hard to trust when the reader cannot see where it came from.

1.2 Approach

Retrieval-augmented generation addresses both problems. Instead of relying on what the model memorised, the system first retrieves the passages of the user's documents most relevant to the question, then asks the model to answer only from those passages. Folio

adds a verification layer on top: each claim is tied to a numbered excerpt, and the excerpt is tied to a file and page the user can open.

1.3 Objectives

1. Let a user upload PDFs into separate notebooks and ask natural-language questions about them.
2. Ground every answer in the uploaded text, and say so plainly when the text does not contain the answer.
3. Attach page-level citations that open the original PDF with the cited passage highlighted.
4. Keep each user's documents and conversations private to their account.
5. Run entirely on free services, deployable on a single machine with one command.

1.4 Scope

In scope: PDF sources with an extractable text layer, English-language retrieval, one chat thread per notebook, and single-machine deployment. Out of scope: scanned documents requiring OCR, other source types (web pages, video transcripts), password recovery by email, sharing between users, and multi-server scaling.

1.5 Success criteria

Criterion	Measure
Working deployment	`docker compose up --build` from a fresh clone serves the app at `http://localhost:8080`
Complete user journey	Register, create a notebook, upload a PDF, see it reach "ready", ask a question, receive a streamed cited answer, open the cited page
Persistence	Notebooks, documents and chat history survive a page reload and a container restart
Isolation	Every attempt by one user to read, list, download, chat with or delete another user's data returns 404
Robustness	Rate limits, interrupted streams, scanned PDFs and invalid uploads produce clear messages instead of failures

2. System overview

A user's work is organised into notebooks: each notebook holds a set of PDF sources and one ongoing conversation, and answers draw only on that notebook's sources.

2.1 User journey

1. **Sign in.** The user creates an account with a username and password, or signs in to an existing one.
2. **Create a notebook.** The home page lists the user's notebooks with their source counts and last-edited times.
3. **Add sources.** The user drags PDFs into the Sources panel. Each file shows "Reading and indexing…" and then its page count, or an error such as "No readable text found, even with OCR.".
4. **Ask.** The user types a question or picks a suggested starter. The answer appears word by word, with numbered highlighter-style chips after each claim and a list of sources beneath.
5. **Verify.** Hovering a chip shows the file, page and snippet. Clicking it opens the PDF viewer at that page and highlights the passage.
6. **Follow up.** Follow-up questions such as "what about phase two?" use the earlier conversation. The history is still there after reloading or signing in again.

2.2 Feature list

Area	Capability
Accounts	Registration and sign-in, 7-day sessions, sign-out, lock-out after 10 failed attempts in 15 minutes
Notebooks	Create, rename, delete; per-notebook sources and chat; newest-edited first
Sources	Multi-file drag-and-drop upload up to 50 MB each, background indexing with live status, OCR for scanned pages, delete
Chat	Streamed answers formatted as markdown (tables, lists, code blocks), citations on every claim, copy answer, suggested starter questions, follow-up context from the last 6 messages, clear chat, stop generating
Verification	Citation hover previews, PDF viewer with page navigation and passage highlighting

Area	Capability
Resilience	Rate-limit countdown with retry, interrupted-response recovery, scanned-PDF detection, recovery of uploads interrupted by a restart
Interface	Light and dark themes, three-panel desktop layout, tabbed mobile layout, keyboard support
Study	Multiple-choice quizzes (5/10/15) and flashcard decks (10/20/30) from chosen sources and an optional topic; every item cites its page; sets and quiz scores saved per notebook

3. System architecture

Folio is three containers behind a single address: nginx serves the React app and forwards `/api` requests to a FastAPI backend, which owns all state in PostgreSQL and a file volume and calls Groq only to write answers.

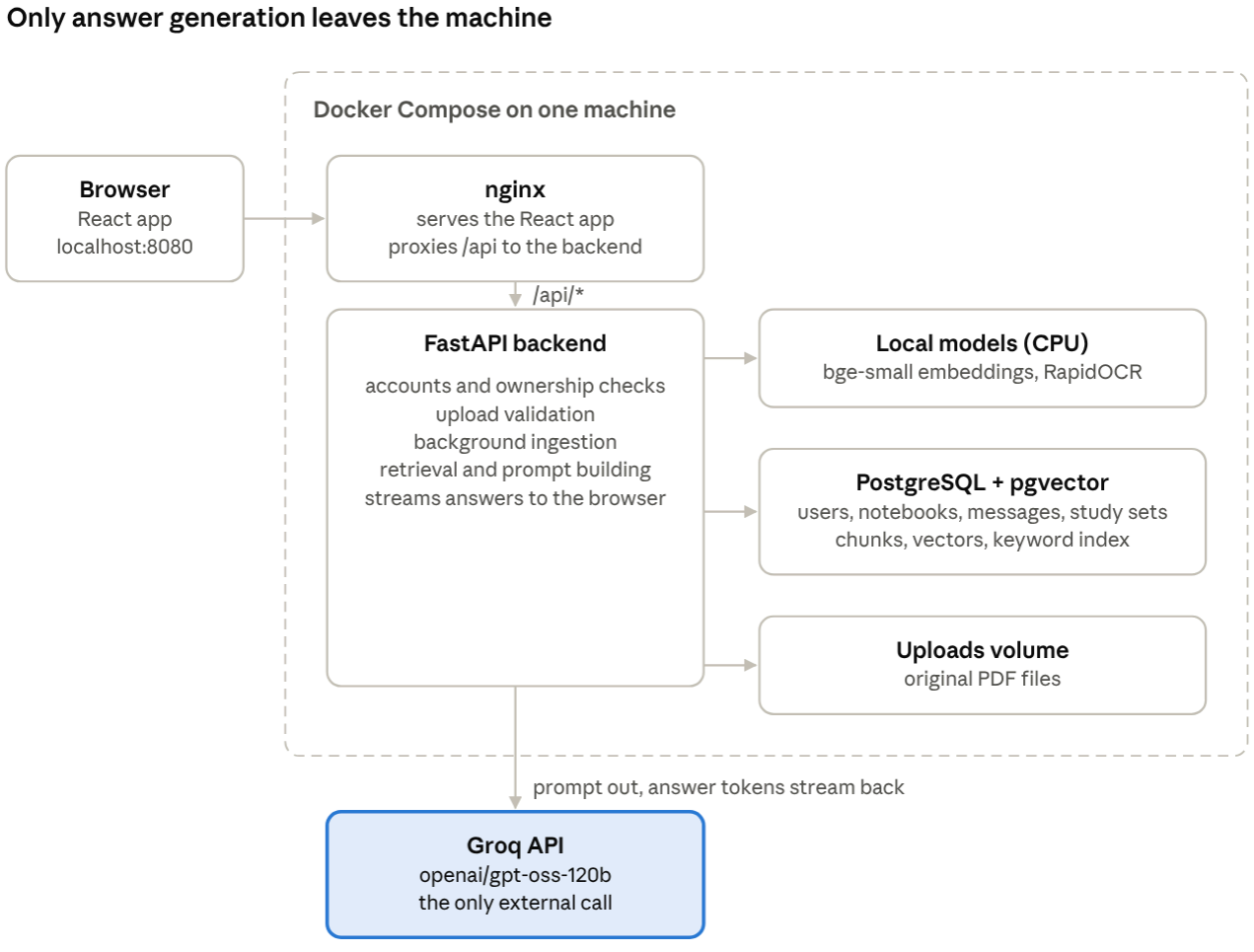


Figure 1 · system architecture: 3 containers, 1 external service

nginx is the single entry point; only the backend touches the database, the PDF files, the embedding model and Groq.

3.1 Component responsibilities

Component	Responsibility	Talks to
Browser (React SPA)	Screens, client-side routing, chat streaming, PDF rendering and highlighting	nginx, same origin
nginx (frontend container)	Serves the built static files; proxies `/api/*` to the backend with buffering disabled so answers stream; falls back to `index.html` for client routes	Backend on the internal network
FastAPI backend	Authentication, ownership checks, upload validation, background ingestion, retrieval, prompt construction, streaming, persistence	PostgreSQL, uploads volume, embedding model, Groq
Embedding model	`bge-small-en-v1.5` loaded once per process; turns chunks and questions into 384-dimension vectors on the CPU	In-process
PostgreSQL 16 + pgvector	Users, notebooks, documents, chunk text with vectors, messages; vector similarity search	Backend only
Uploads volume	Original PDF files, stored by user and document id	Backend only
Groq API	Hosted LLM (`openai/gpt-oss-120b`) that generates the streamed answer	Backend, outbound HTTPS

3.2 Design properties

- **Same origin.** Because nginx serves both the app and the API, the session cookie needs no cross-origin configuration and can be `httpOnly` and `SameSite=Lax`.

- **One external dependency.** Only answer text generation leaves the machine. Parsing, chunking, embedding and search run locally, so the high-volume work costs nothing.
- **Stateless web tier.** All durable state lives in two Docker volumes, `pgdata` and `uploads`; containers can be rebuilt or restarted without data loss.

4. Technology stack and rationale

Every choice favours free, widely used, single-machine tooling; the one hosted service (Groq) was chosen because it offers a capable model on a free tier.

Layer	Technology	Role	Why this over alternatives
Backend language	Python 3.13	API and ML pipeline	Direct access to the ML ecosystem (sentence-transformers, PyMuPDF); Go or Node would need bridges to it
Web framework	FastAPI	HTTP API, dependency injection, validation	Native async for streaming and concurrent I/O; Pydantic validation; generated OpenAPI docs at `/docs`
Database	PostgreSQL 16 + pgvector 0.5	Relational data and vector search	One database for both workloads; joins between vectors and ownership tables; avoids running a separate vector store
Vector index	HNSW (cosine)	Approximate nearest-neighbour search	Accurate from the first row; IVFFlat built on a near-empty table returns too few results
ORM and migrations	SQLAlchemy 2 (async) + asyncpg, Alembic	Data access, versioned schema	Typed models, non-blocking queries, reproducible schema changes

Layer	Technology	Role	Why this over alternatives
PDF parsing	PyMuPDF	Per-page text extraction	Fast, reliable text layer extraction that keeps page boundaries, which citations depend on
Embeddings	sentence-transformers, `BAAI/bge-small-en-v1.5`	384-dimension text vectors	Free and local; strong retrieval quality for its size (~130 MB); runs on CPU
LLM	Groq, `openai/gpt-oss-120b`	Answer generation, streamed	Free tier with a large open-weight model and fast streaming
Auth	bcrypt, PyJWT	Password hashing, signed sessions	Standard, audited primitives; stateless sessions need no session table
Frontend	React 19, TypeScript, Vite	Single-page app	Component model suited to a three-panel workspace; type safety across API types
Styling	Tailwind CSS 4	Design tokens and layout	Theme variables for light and dark mode without a component library
Data fetching	TanStack Query	Server state, caching, polling	Cache updates after mutations and status polling while documents process
PDF viewer	react-pdf (pdf.js)	Page rendering and text layer	Renders pages in the browser and exposes text items for highlighting
Delivery	Docker Compose, nginx	Packaging and serving	One command to run; nginx handles static files, proxying and streaming

Layer	Technology	Role	Why this over alternatives
Answer rendering	react-markdown, remark-gfm	Markdown answers: tables, headings, lists, code	Full GitHub-flavoured markdown; builds React elements, so model output is never injected as HTML
Keyword search	PostgreSQL full-text search (tsvector, GIN)	Exact-term retrieval fused with vector search	Built into the existing database; no extra search service
OCR	RapidOCR (rapidocr-onnxruntime)	Text from scanned PDF pages	Pure pip install with bundled models, works offline and on Windows; Tesseract needs a separate system install

5. Data model

Six tables form a strict ownership chain — user, notebook, then documents, messages and study sets, then chunks — so every row can be traced to exactly one user, and deleting a parent removes everything beneath it.

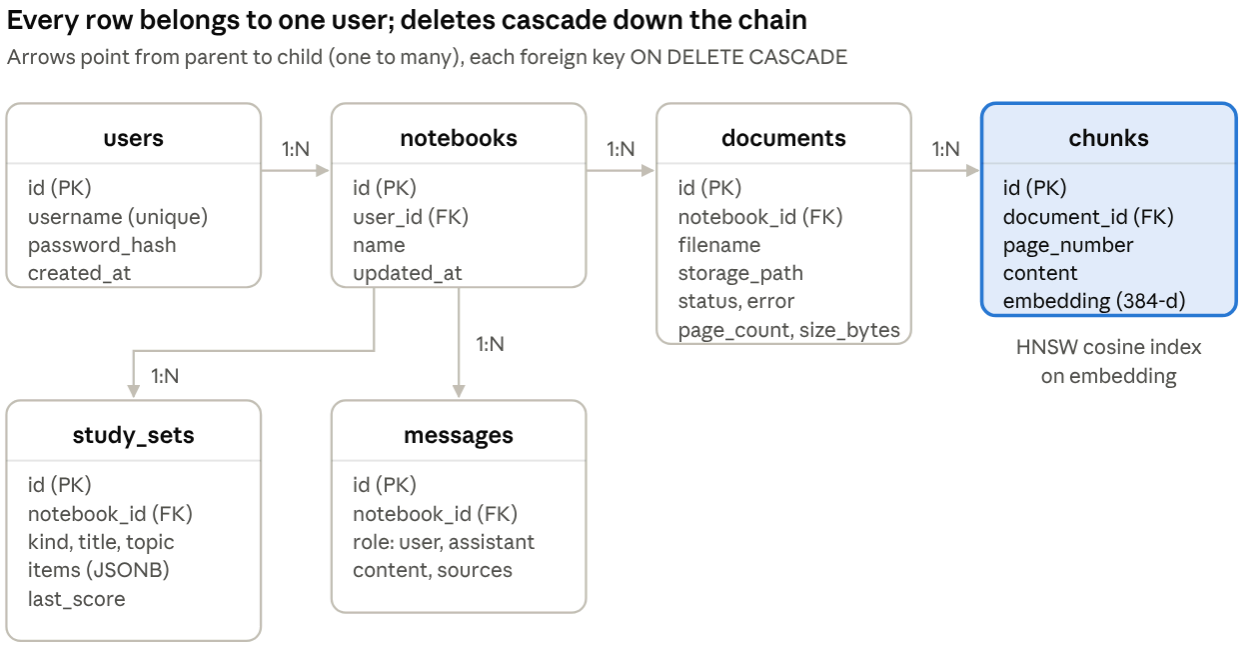


Figure 2 · entity-relationship diagram: 6 tables

Chunks are highlighted because they are what retrieval searches and what every citation points to.

Table	Key columns	Notes
`users`	`id` (UUID), `username` (unique, lowercase, 3–32 of `a-z 0-9 _`), `password_hash`, `created_at`	Password stored only as a bcrypt hash
`notebooks`	`id`, `user_id` → users, `name` (≤ 120 chars), `created_at`, `updated_at`	`updated_at` moves on rename, upload, delete and each saved message; drives home-page ordering
`documents`	`id`, `notebook_id` → notebooks, `filename`, `storage_path`, `size_bytes`, `page_count`, `status`, `error`, `created_at`	`status` is `processing`, `ready` or `failed`; `storage_path` is built from ids, never from the uploaded name
`chunks`	`id`, `document_id` → documents, `page_number`, `chunk_index`, `content`, `embedding vector(384)`	The unit of retrieval and citation; content_tsv is a generated full-text column for keyword search
`messages`	`id`, `notebook_id` → notebooks, `role` (`user` or `assistant`), `content`, `sources` (JSONB), `created_at`	`sources` stores the resolved citations with each answer
study_sets	id, notebook_id → notebooks, kind (quiz or flashcards), title, topic, document_ids (JSONB), items (JSONB), last_score, created_at	Items store question data with each citation resolved to file and page

Indexes. Each foreign key is indexed (`ix_notebooks_user_id`, `ix_documents_notebook_id`, `ix_chunks_document_id`), messages use a composite index on `(notebook_id, created_at)` for ordered history, and `ix_chunks_embedding_hnsw` is an HNSW index with `vector_cosine_ops` for similarity search, and ix_chunks_content_tsv is a GIN index over the generated content_tsv column for keyword search.

Deletion. Every foreign key is declared `ON DELETE CASCADE`, and the ORM relationships use `passive_deletes`, so the database removes thousands of chunks in one statement rather than the application loading and deleting them row by row. PDF files on disk are removed by the application after the database commit succeeds.

Migrations. The schema is created and versioned by Alembic (`alembic/versions/0001_initial.py`), which also enables the `vector` extension. Migration 0002 adds the keyword-search column, the OCR page count and the study_sets table. The backend container applies pending migrations on every start.

6. Document ingestion pipeline

An upload is accepted in one request and indexed in the background, so the user never waits on a long-running HTTP call; the document's status field reports progress.

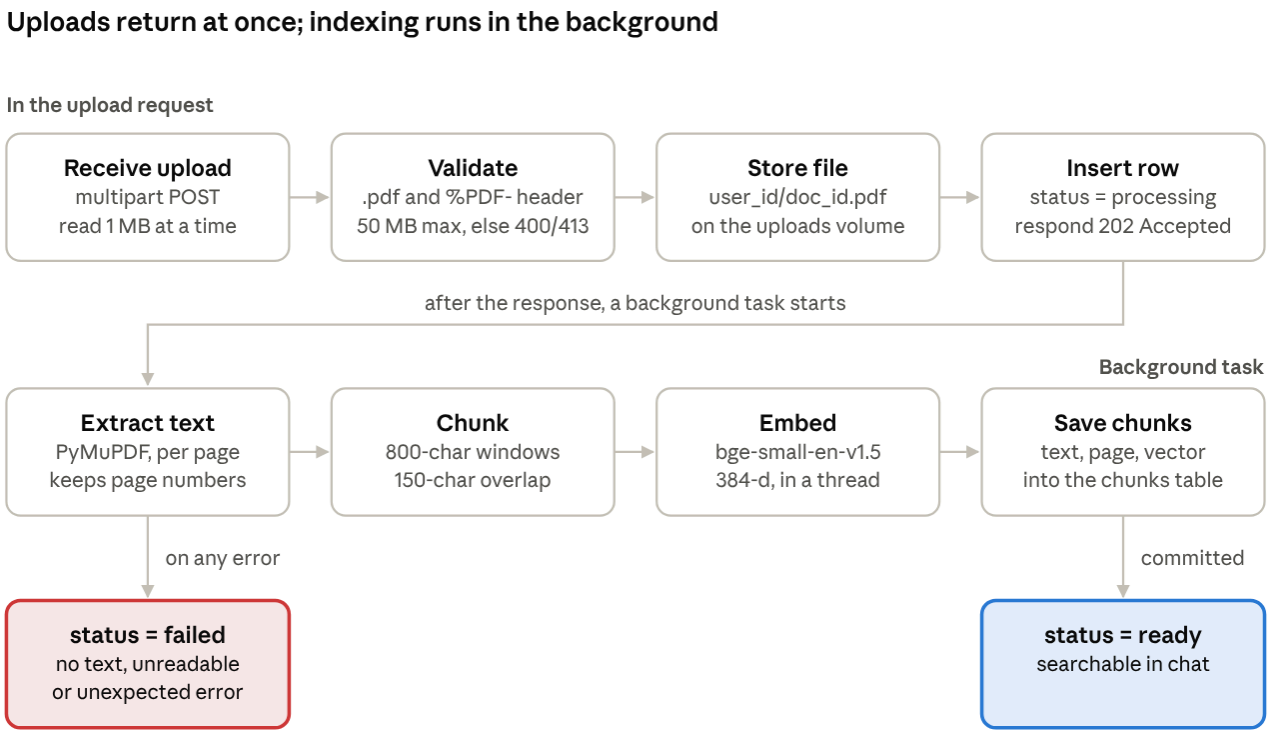


Figure 3 · ingestion pipeline: 4 request steps, 4 background steps

The browser polls the document's status, so the user sees processing turn into ready or failed without refreshing.

6.1 Stages

1. **Validate** (`routers/documents.py`). The filename must end in `.pdf`, the bytes must begin with the `%PDF-` signature, and the upload is read in 1 MB pieces and rejected with HTTP 413 the moment it passes `MAX_UPLOAD_MB` (50).
2. **Store** (`services/storage.py`). The file is written to `UPLOAD_DIR/<user_id>/<document_id>.pdf`. Paths are resolved and checked to stay inside the upload directory.
3. **Register.** A `documents` row is inserted with status `processing`, the notebook's `updated_at` moves, and the API responds `202 Accepted` with the new document.

4. **Extract** (`services/pdf_parser.py`). A background task opens the file with PyMuPDF and reads each page's text. A page whose text layer has fewer than 20 characters is treated as scanned: it is rendered at 200 DPI and read with RapidOCR, a local CPU OCR engine whose models ship inside the package. Pages with no text even after OCR are skipped, the true page count is kept, and the number of OCR'd pages is stored for display.
5. **Chunk** (`services/chunking.py`). Each page is cut into windows of 800 characters that advance by 650, so neighbouring chunks overlap by 150 characters and a sentence split at one boundary appears whole in the next chunk. Every chunk keeps its page number; chunks never span two pages.
6. **Embed** (`services/embeddings.py`). All chunks are encoded in one batch by `bge-small-en-v1.5` with L2 normalisation, in a worker thread so the server keeps answering other requests.
7. **Persist** (`services/ingest.py`). Chunks and vectors are inserted, `page_count` is set and the status becomes `ready` .

6.2 Status lifecycle and failure handling

From	Event	To	Message shown
(new)	Upload accepted	`processing`	Reading and indexing…
`processing`	Chunks stored	`ready`	Page count and file size
`processing`	No page has text, even after OCR	`failed`	No readable text found, even with OCR.
`processing`	File cannot be opened as a PDF	`failed`	This file couldn't be read as a PDF.
`processing`	Unexpected error	`failed`	Processing failed — please try uploading again.
`processing`	Server restarted mid-job (detected at startup)	`failed`	Processing was interrupted — please re-upload.

The frontend polls the documents list every 2 seconds while any document is `processing` and stops when none are. If a document is deleted while it is being embedded, the job notices and discards its results.

7. Query and answer generation pipeline

A question becomes a streamed, cited answer in seven steps; tokens are forwarded the moment Groq produces them, and complete answers took between one and seven seconds in testing.

The answer streams to the browser token by token while Groq writes it

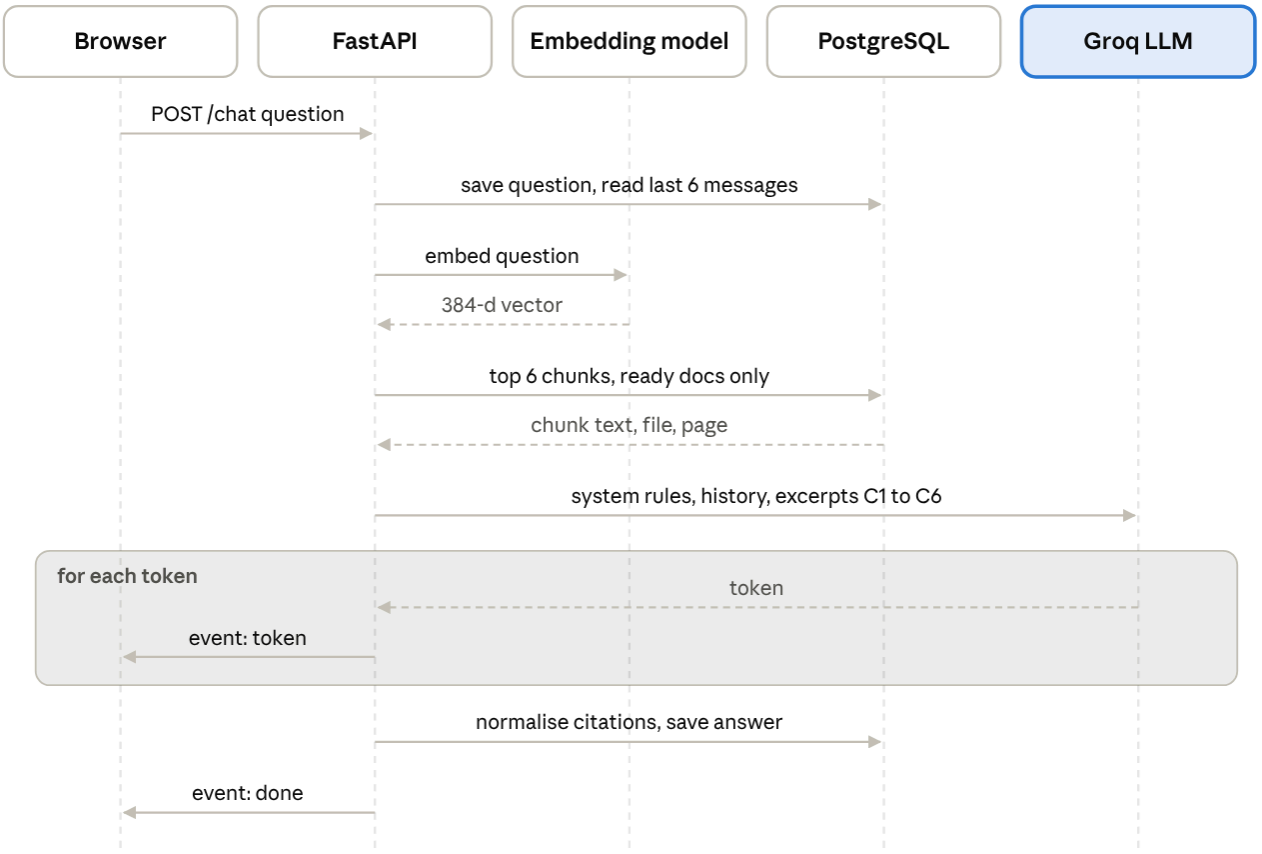


Figure 4 · sequence of one question: 5 participants, 1 streaming loop

Solid arrows are requests and dashed arrows are replies; the shaded loop repeats for every token Groq produces.

7.1 Steps

1. **Record the question** (`routers/chat.py`). The question (1–2,000 characters) is saved as a `user` message. If the notebook's last message is the same unanswered question — a retry after an error — that row is reused instead of duplicated.
2. **Load history.** The six messages before the question are read so the model can resolve follow-ups ("how many engineers are on *their* team?"). Old citation markers are stripped from them because they refer to excerpts that are not in the new prompt.
3. **Embed the question** with the same model used for chunks, in a worker thread.

4. **Retrieve.** Two searches run over this notebook's `ready` documents: the 30 chunks closest to the question by cosine distance, and the 30 best full-text matches for the question's words. Reciprocal rank fusion merges the two rankings into the final six.
5. **Build the prompt** (`services/llm.py`). A system message sets the rules; history follows as prior turns; the final user turn holds the numbered excerpts and the question.
6. **Generate and stream.** Groq streams tokens; each is forwarded to the browser immediately as a Server-Sent Event (`event: token`).
7. **Finalise.** The full text is normalised, its citation markers are resolved into sources, the `assistant` message is saved with them, and a final `done` event carries the saved message.

7.2 Retrieval scoring

Chunk and question vectors are L2-normalised, so ranking by cosine distance is equivalent to ranking by dot product:

$$\text{distance}(q, c) = 1 - \frac{q \cdot c}{\lVert q \rVert \, \lVert c \rVert}, \qquad \text{retrieved} = \operatorname{arg\,top}_{k=6} \; \bigl(-\text{distance}(q, c)\bigr)$$

Keyword search uses a generated `tsvector` column (English stemming, GIN index) queried with the question's terms OR-ed together, so one exact term such as a course code is enough to match. The two ranked lists are fused by reciprocal rank fusion, where k = 60 damps the weight of the very top ranks:

$$\text{score}(c) = \sum_{r \,\in\, \{\text{vector},\,\text{keyword}\}} \frac{1}{60 + \text{rank}_r(c)}$$

On a test handbook of 16 near-identical course codes (BCSE301L to BCSE316L), vector search alone ranked the correct page first for 15 codes; hybrid search did so for all 16.

7.3 Prompt structure

Part	Content
System message	Answer only from the excerpts and say so when they don't contain the answer; cite every claim as `[[C1]]` in plain ASCII brackets; never invent markers; use history only to understand follow-ups; be concise

Part	Content
History	Up to 6 prior user and assistant turns, citation markers removed
User turn	`[C1] (from "report.pdf", page 4)` followed by the chunk text, for C1 to C6, separated by rules; then `Question: …`

7.4 Edge cases

- **No ready sources.** The model is not called; a fixed reply asks the user to upload a source.
- **Rate limit (HTTP 429 from Groq).** The stream ends with `event: error` carrying `retry_after` from Groq's header; no assistant message is saved, so a retry produces a clean answer.
- **Bad key, Groq outage, network loss.** Each maps to a specific user-facing message in the `error` event.
- **Client disconnects.** The generator is cancelled; the question stays saved and the history resynchronises on the next load.
- **Token budget.** `max_tokens` is 4,096 because gpt-oss spends part of its budget on internal reasoning before writing the answer.

7.5 Quiz and flashcard generation

Study sets reuse the same excerpt labels and citation map as chat, but ask for structured JSON instead of prose, within Groq's free-tier limit of 8,000 tokens per minute.

1. **Request.** The user chooses quiz (5, 10 or 15 questions) or flashcards (10, 20 or 30 cards), the sources to include (default: all ready sources) and an optional focus topic.
2. **Select excerpts.** With a topic, hybrid search returns up to 18 matching chunks; without one, chunks are taken at even intervals across the selected sources so the set covers them start to end. Selection stops at a character budget: 12,000 for quizzes and 16,000 for flashcards.
3. **Generate.** One non-streamed call in JSON mode asks for exactly the requested number of items, each citing one excerpt label. Quizzes use medium reasoning effort and flashcards low, because measured usage for 15 questions was 2,266 tokens at low effort and 4,318 at medium, and a low-effort quiz produced a wrong answer key.
4. **Validate.** Items are checked with Pydantic and dropped if malformed, if a quiz item lacks four distinct options or a valid answer index, or if the cited label does not exist. Quiz options are shuffled so the correct answer's position is uniform, leaked excerpt

labels are rewritten as "the source", and each citation is resolved to file, page and snippet.

5. **Save.** If at least half the requested items survive, the set is stored in `study_sets`; otherwise the API returns 502 "Couldn't generate a good set — try again". Malformed JSON gets one retry, and a Groq rate limit returns 429 with the wait time.

The prompt forbids negative questions ("which is NOT"), options that differ only in wording or order, and "all of the above". Generation took 2 to 7 seconds in testing.

8. Citation mechanism

Citations are treated as data, not prose: each excerpt gets a short label, the model writes labels after its claims, and the system turns labels back into file-and-page references the interface can open.

8.1 From excerpt to marker

While building the prompt, `build_context()` labels the retrieved chunks `C1` to `C6` and records a citation map from each label to `{marker, chunk_id, document_id, filename, page_number, snippet}`, where the snippet is the chunk's first 240 characters. The model is told to write `[[C1]]` after each claim it takes from excerpt C1.

8.2 Normalising what the model actually writes

Models do not always follow the requested format. During testing, gpt-oss consistently wrote its native style `【C1】` instead of `[[C1]]`, which at first left every answer without sources. `normalize_citations()` now rewrites every observed variant to the canonical form before parsing:

Model output	Normalised
`[[C1]]`	`[[C1]]`
`[C3]`	`[[C3]]`
`[[C1, C2]]`	`[[C1]][[C2]]`
`【C2】`	`[[C2]]`
`【C1†L3-L5】`	`[[C1]]`
`[citation needed]`, `[1]`	unchanged (not a marker)

The same rules run in the browser while an answer is still streaming, and a marker cut off mid-stream (such as a trailing `[[C`) is hidden until it completes.

8.3 From marker to source

`parse_citations()` collects the distinct marker numbers in the answer and looks each up in the citation map. Markers with no entry — an invented `[[C9]]` — are dropped rather than shown as broken links. The resolved sources are stored in the message's `sources` column with their `marker`, so a saved answer maps each marker to its source exactly when it is reloaded.

8.4 Display numbering

The interface renumbers citations by first appearance, so an answer that cites C3 and then C1 shows chips 1 and 2 in reading order. Repeated adjacent chips (`[[C1]][[C1]]`) collapse into one. Before rendering, each marker is rewritten as a markdown link to a reserved #cite- address, so a chip can appear anywhere markdown allows a link, including table cells, list items and headings. Each chip is a button with an accessible label such as "Source 1: report.pdf, page 4", a hover card with the snippet, and an entry in the source list under the answer.

8.5 Highlighting the passage in the PDF

The backend extracted the chunk with PyMuPDF, but the browser renders the page with pdf.js, and the two split whitespace and line breaks differently, so a plain text search for the snippet fails. The viewer therefore:

1. Concatenates the page's text-layer items after removing all whitespace and lowercasing, recording where each item starts.
2. Searches that string for the snippet, prepared the same way; if the full snippet is not found it tries its first 60 and then its last 60 characters.
3. Wraps every text item that overlaps the match in a `<mark>` element, escaping the item text, and scrolls the first mark into view.

If nothing matches, the viewer still opens the cited page without a highlight. This is always the case for OCR'd pages, which have no text layer to highlight.

9. Authentication, security and data isolation

Every request outside `/auth` and `/health` must carry a valid session, and every resource lookup is filtered by the signed-in user, so one user can neither see nor confirm the existence of another user's data.

9.1 Authentication

- **Passwords** are 8–128 characters and stored as bcrypt hashes with a per-password salt. Login with an unknown username still runs a bcrypt comparison against a dummy hash, so response time does not reveal which usernames exist.
- **Usernames** are trimmed and lowercased before validation, so `Alice` and `alice` are the same account.
- **Sessions** are JWTs signed with HS256 using `JWT_SECRET`, containing the user id (`sub`) and a 7-day expiry. They travel in a cookie named `session` marked `httpOnly` (unreadable by page scripts, limiting cross-site scripting), `SameSite=Lax` (not sent on cross-site form posts, limiting request forgery) and optionally `Secure`.
- **Startup check.** The backend refuses to start if `JWT_SECRET` is unset or still the placeholder `change-me`, so it can never sign tokens with a publicly known key.
- **Brute-force protection.** Ten failed logins for the same username from the same IP address within 15 minutes block further attempts with HTTP 429 and a `Retry-After` header, even with the correct password. Behind nginx, the client address comes from the `X-Real-IP` header nginx sets.

9.2 Data isolation

All lookups go through three dependencies in `app/deps.py`: `get_current_user`, `get_owned_notebook` and `get_owned_document`. The document check joins through `notebooks.user_id`, and retrieval only searches the requested, already-verified notebook. A resource that exists but belongs to someone else returns **404 Not Found**, never 403, so identifiers cannot be probed. Testing with two accounts confirmed 404 for another user's notebook read, rename, delete, upload, document metadata, PDF download, chat and history, and for listing, opening, scoring, deleting and generating study sets.

9.3 Upload and file safety

- Extension and `%PDF-` signature checks reject non-PDF content even when renamed.
- Uploads are read in 1 MB pieces and stopped at the limit, so an oversized file cannot exhaust memory.
- Files are stored under names made from UUIDs; the user-supplied filename is kept only as display data, and resolved paths are checked to stay inside the upload directory.
- PDF downloads are served only through the ownership-checked endpoint with a private cache header.

9.4 Secrets and transport

The Groq key, JWT secret and database password live in `.env`, which is excluded from Git and from Docker build contexts. In Docker, the backend is reachable only on the internal network and the frontend is the application's single entry point. PostgreSQL's port 5432 is also published to the host for local development; that mapping should be removed before any public hosting. Serving over HTTPS requires setting `COOKIE_SECURE=true`.

9.5 Rendering safety

Answers are rendered as GitHub-flavoured markdown by react-markdown, which builds React elements rather than injecting HTML. Raw HTML in model output is dropped, except
 line breaks inside table cells, which are converted to markdown line breaks; an answer containing a was verified to render nothing and run nothing. Text placed into the PDF text layer is HTML-escaped before it is wrapped in highlight marks.

10. API reference

The REST API lives under `/api`, exchanges JSON (multipart for uploads), authenticates with the session cookie, and reports errors as `{"detail": "…"}`; interactive documentation is generated at `/docs`.

10.1 Endpoints

Method	Path	Purpose	Success	Notable errors
POST	`/api/auth/register`	Create account and sign in	201 + user	409 username taken, 422 invalid input
POST	`/api/auth/login`	Sign in	200 + user	401 wrong credentials, 429 locked out
POST	`/api/auth/logout`	Clear the session cookie	204	—
GET	`/api/auth/me`	Current user	200	401
GET	`/api/notebooks`	List own notebooks with source counts, newest first	200	401

Method	Path	Purpose	Success	Notable errors
POST	`/api/notebooks`	Create notebook	201	422 empty name
GET	`/api/notebooks/{id}`	One notebook	200	404
PATCH	`/api/notebooks/{id}`	Rename	200	404, 422
DELETE	`/api/notebooks/{id}`	Delete with its sources, files and chat	204	404
GET	`/api/notebooks/{id}/documents`	List sources with status	200	404
POST	`/api/notebooks/{id}/documents`	Upload a PDF (multipart field `file`)	202, status `processing`	400 not a PDF, 413 too large, 404
GET	`/api/documents/{id}`	Source status, for polling	200	404
GET	`/api/documents/{id}/file`	Original PDF, inline	200 `application/pdf`	404
DELETE	`/api/documents/{id}`	Remove source, chunks and file	204	404
GET	`/api/notebooks/{id}/messages`	Chat history, oldest first	200	404
DELETE	`/api/notebooks/{id}/messages`	Clear chat	204	404
POST	`/api/notebooks/{id}/chat`	Ask a question; streamed response	200 `text/event-stream`	404, 422
GET	`/api/health`	Liveness check	200	—
GET	/api/notebooks/{id}/study-sets	List quizzes and flashcard decks, newest first	200	404
POST	/api/notebooks/{id}/study-sets	Generate a quiz or deck: kind, count, topic?, document_ids?	201 + set	400 no ready source, 422 bad count, 429 rate limited, 502 generation failed
GET / DELETE	/api/study-sets/{id}	Open or delete a set	200 / 204	404

Method	Path	Purpose	Success	Notable errors
POST	/api/study-sets/{id}/score	Save a quiz score	200	400 not a quiz, 422 out of range, 404

10.2 Streaming protocol

`POST /chat` with body `{"question": "…"}` returns Server-Sent Events. The browser reads them with `fetch` and a stream reader, because the standard `EventSource` API cannot send a POST body.

Event	Data	Meaning
`token`	`{"text": "…"}`	Next piece of the answer; repeated
`done`	`{"message": {id, role, content, sources[], created_at}}`	Answer complete and saved; `content` normalised, `sources` resolved
`error`	`{"detail": "…", "retry_after": 12}`	Generation failed; `retry_after` in seconds when rate limited, otherwise null

Example of a complete stream:

```
event: token
data: {"text": "Phase two targets "}

event: token
data: {"text": "satellite imaging [[C1]]."}

event: done
data: {"message": {"id": "…", "role": "assistant", "content": "Phase two
targets satellite imaging [[C1]].", "sources": [{"marker": "C1", "filename":
"report.pdf", "page_number": 3, "snippet": "Phase two of Falcon targets…",
"chunk_id": "…", "document_id": "…"}], "created_at": "…"}}
```

Responses carry `Cache-Control: no-cache` and `X-Accel-Buffering: no` so proxies pass each event through immediately.

11. Frontend design

The interface is a single-page React application with three routes and one central workspace, designed so that checking a claim takes one click from the answer to the highlighted page.

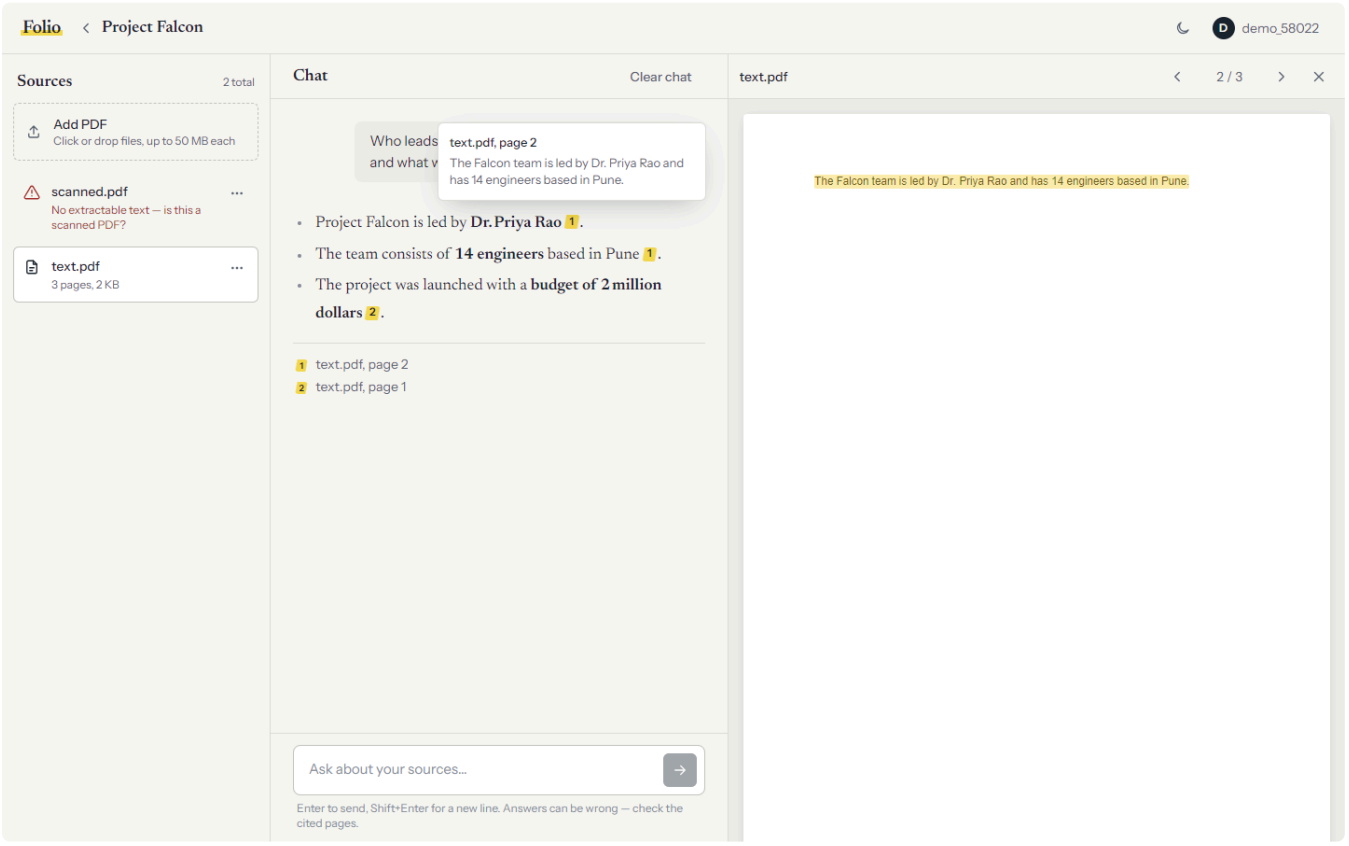


Figure 5 · The workspace: a hover card previews source 1, and the viewer shows page 2 with the cited sentence highlighted. The scanned PDF in the Sources panel shows its failure reason.

11.1 Screens

Route	Screen	Behaviour
`/login`	Sign in / create account	One form that switches mode; inline field hints in plain language; a sample cited answer beside the form shows what the product does
`/`	Notebooks	Card grid with name, source count and last edit; create inline; rename and delete from a card menu, with confirmation
`/notebooks/:id`	Workspace	Sources, Chat and PDF viewer panels

Signed-out visitors are redirected to `/login` and returned to their original page after signing in; a 401 from any request clears cached data and returns the user to sign-in.

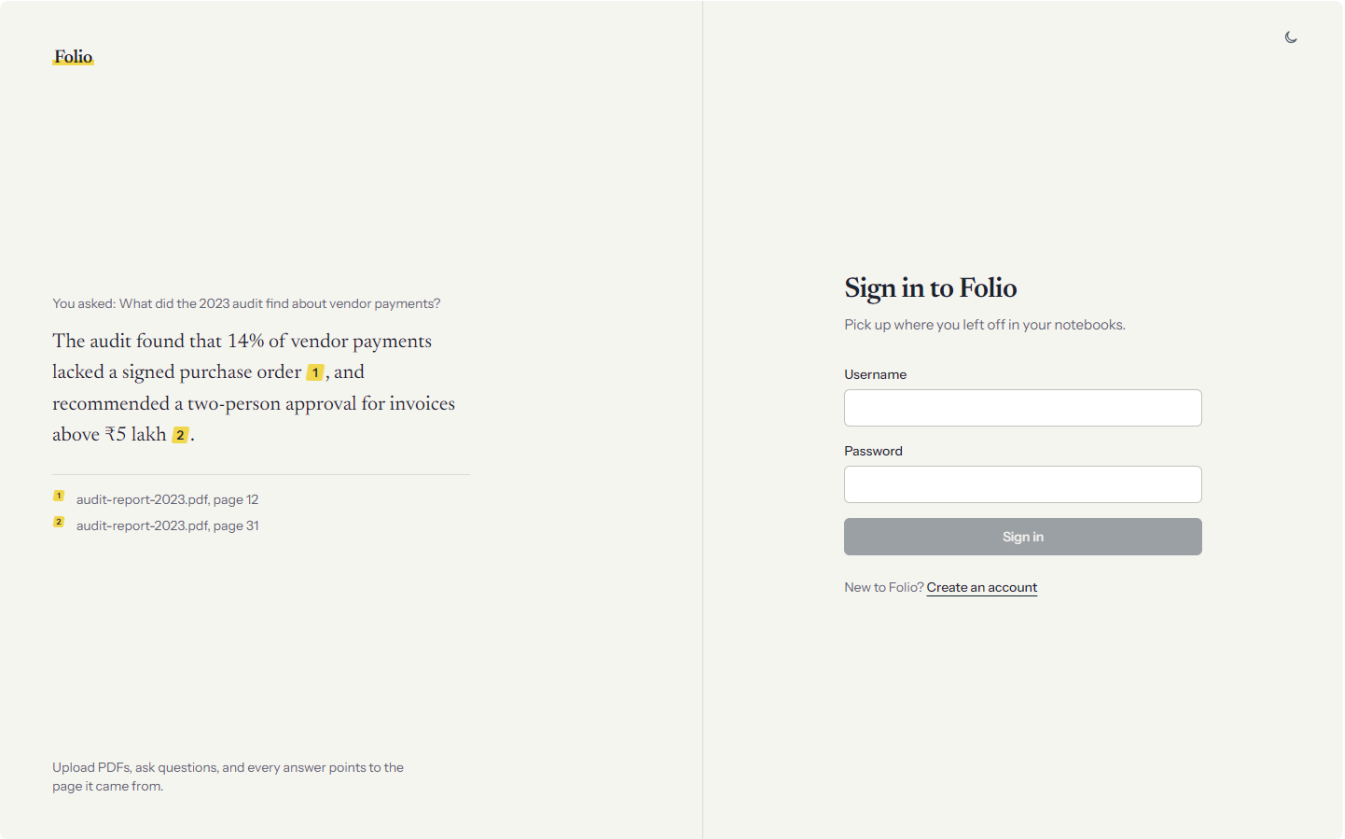


Figure 6 · The sign-in page shows a sample cited answer, so a first-time visitor sees what the product does before creating an account.

11.2 Workspace layout

At 1,024 px and wider the workspace shows three columns: Sources (288 px), Chat (flexible) and the PDF viewer (46% of the width, hidden until a citation or source is opened). Below 1,024 px the panels become tabs, and opening a citation switches to the Viewer tab. The chat panel stays mounted across tab switches so an answer keeps streaming.

11.3 Client architecture

Module	Responsibility
`api/client.ts`	`fetch` wrapper: same-origin cookies, JSON bodies, translation of FastAPI errors into readable messages, global 401 handling
`api/queries.ts`	TanStack Query hooks for every endpoint; cache updates after mutations; 2-second polling while any document is processing

Module	Responsibility
`api/stream.ts`	SSE reader over `fetch`: buffers partial chunks, splits events on blank lines, dispatches `token`, `done` and `error`, reports a stream that ends without either as interrupted
`lib/citations.ts`	Marker normalisation, partial-marker hiding, renumbering by first appearance, dropping unknown markers, rewriting markers as chip links for the markdown renderer
`components/workspace/*`	Sources panel, chat panel, markdown answer renderer (react-markdown with GFM tables, citation chips, copy buttons for answers and code blocks), PDF viewer

The workspace, including the markdown renderer, loads only when a notebook is opened, and the PDF viewer with pdf.js only when a document is first viewed; this keeps the initial JavaScript bundle at 370 KB instead of over 1 MB. Error boundaries contain a viewer failure to its own panel.

11.4 Chat states

- **Empty notebook:** guidance to add a source; the composer is disabled.
- **Ready, no messages:** three starter questions.
- **Streaming:** the question appears at once, the answer grows with a blinking caret, and Send becomes Stop.
- **Rate limited:** the partial answer stays, with the reason, a "Retry in N s" countdown and Dismiss.
- **Interrupted:** partial text with "Response interrupted before it finished" and Retry.

11.5 Visual design

The design follows a highlighter metaphor: citation chips use the same yellow as the highlight drawn on the cited passage, so the link between claim and page is visible without explanation.

Token	Light	Dark	Use
Paper	#F6F6F3	#14171D	Page background
Ink	#1C2330	#E7E9EC	Text, primary buttons

Token	Light	Dark	Use
Muted	#5D6573	#A3A9B3	Secondary text
Line	#DCDDD8	#2C313B	Borders and dividers
Highlight	#F3D74C	#D9BD3A	Citations and PDF highlights only

Headings and answers use Newsreader, a serif designed for screen reading; interface text uses Instrument Sans. The theme follows the system setting, can be switched in the header, and is applied before first paint to avoid a flash. Focus outlines are visible for keyboard users and animations are disabled when the system requests reduced motion.

11.6 Study panel

A Chat | Study switch in the centre panel's header changes what the panel shows; both views stay mounted, so a streaming answer or an open quiz survives switching.

- **Study home** lists saved sets with type, size, age, focus topic and the last quiz score, with New quiz, New flashcards and Delete.
- **Generate form**: source checkboxes, focus topic, count selector; a progress state while the set is written; on a rate limit, the error and a "Retry in N s" countdown.
- **Quiz player**: one question at a time with four lettered options; picking one marks it right or wrong, reveals the correct answer, a one-line explanation and a citation chip that opens the PDF page; the score is saved at the end and the quiz can be retaken without using quota. Keys 1–4 answer and Enter advances.
- **Flashcard player**: a card that flips in 3D (instantly when reduced motion is requested), previous and next, shuffle, and the citation chip on the back. Space flips and the arrow keys move; shortcuts are ignored while the Study view is hidden.

12. Deployment and operations

The whole system starts with `docker compose up --build` and needs only a Groq API key and a session secret; after the first build it needs no internet access except for calls to Groq.

12.1 Containers

Service	Image	Exposed	Persistent data	Start condition
`db`	`pgvector/pgvector:pg16`	5432 (local development)	`pgdata` volume	Health check `pg_isready`

Service	Image	Exposed	Persistent data	Start condition
`backend`	Built from backend/ `Dockerfile` (Python 3.13 slim)	Internal 8000 only	`uploads` volume at `/data/uploads`	Waits for a healthy `db`; health check on `/api/health`
`frontend`	Built from `frontend/Dockerfile` (Node 22 build, nginx 1.29 serve)	8080 → 80	None	Waits for a healthy `backend`

12.2 Build decisions

- **CPU-only PyTorch.** The default Linux PyTorch wheels bundle about 2.5 GB of NVIDIA GPU libraries. A Linux-only package source in backend/ `pyproject.toml` installs `torch 2.14.1+cpu` instead; with the OCR engine and the OpenCV system libraries it needs (libgl1, libglib2.0-0), the finished backend image is 3.18 GB, still with no NVIDIA packages.
- **Model baked into the image.** The embedding model (129 MB) is downloaded during the build and the container runs with `HF_HUB_OFFLINE=1`, so it never contacts Hugging Face at runtime.
- **Reproducible dependencies.** Python packages install from `uv.lock` with `uv sync --frozen`, and frontend packages from `package-lock.json` with `npm ci`.
- **Least privilege.** The backend runs as an unprivileged `folio` user.
- **nginx for streaming.** The `/api` proxy disables buffering and allows 300-second reads so answers stream token by token; `client_max_body_size` is 55 MB to admit 50 MB uploads; hashed assets are cached for a year; the `.mjs` pdf.js worker is served as JavaScript.

12.3 Startup sequence

1. PostgreSQL starts and reports healthy.
2. The backend entrypoint runs `alembic upgrade head`, applying any pending migrations.
3. On application start it checks `JWT_SECRET`, marks any document left in `processing` as failed (interrupted), and loads the embedding model so the first upload or question is not slow.
4. Once `/api/health` answers, nginx starts serving on port 8080.

12.4 Configuration

Settings come from the `.env` file at the repository root (see the Appendix). Docker Compose overrides `DATABASE_URL` to point at the `db` service and sets `UPLOAD_DIR` to the volume, so the same file works both for Docker and for running the backend directly from `backend/`.

12.5 Development mode

Run only the database in Docker (`docker compose up -d db`). Then, inside `backend/`, run `uv sync`, `uv run alembic upgrade head` and `uv run uvicorn app.main:app --port 8000`; in `frontend/`, run `npm run dev`. Vite serves the app on port 5173 with hot reload and forwards `/api` to the backend, preserving same-origin cookies. The backend reads the `.env` at the repository root.

12.6 Hosting options

Option	Code changes	Availability	Fit
Laptop + Cloudflare Tunnel	None	While the laptop runs	Live demonstrations
Oracle Cloud Always Free ARM VM with Docker	None; add HTTPS and `COOKIE_SECURE=true`	Always on	Persistent public deployment
Hugging Face Spaces + Neon or Supabase Postgres	Move PDF storage to object storage; listen on port 7860	Sleeps when idle	Free cloud hosting with external database

Free platforms with 512 MB of memory are unsuitable because PyTorch and the embedding model need about 1.5–2 GB.

13. Verification and results

The system was verified end to end rather than with a unit-test suite: every feature was exercised against a running stack, first in development and then on the production Docker build, and all scenarios below passed on the final build.

13.1 Method

- **API level.** Scripted HTTP requests with two separate accounts covered authentication, validation, ownership, uploads, processing and streaming.

- **Browser level.** Automated Chrome sessions (Playwright) performed the full user journey at 1,440 × 900 and at a 390 × 844 phone size, capturing screenshots at each step.
- **Failure injection.** The browser's network layer was intercepted to return a rate-limit error, a stream cut off mid-answer, and answers containing invalid and non-standard citation markers.
- **Production build.** The same browser scenarios ran against a fresh, isolated Docker Compose stack, which was then restarted to check persistence.
- **Test documents.** A three-page text PDF with known facts, an image-only PDF, a text file renamed `.pdf`, and a 51 MB file.

13.2 Scenarios

Area	Scenario	Result
Auth	Register, reload (session kept), sign out, sign back in	Pass
Auth	Duplicate username in different case; short password; invalid characters	409; 422 with plain-language hint; 422
Auth	11 wrong passwords, then the correct one	10 × 401, then 429 even for the correct password
Auth	Placeholder `JWT_SECRET`	Startup refused with instructions
Isolation	Second user reads, renames, deletes, uploads to, downloads from, or chats in the first user's notebook	404 in every case; their notebook list is empty
Upload	Text PDF	processing → ready, 3 pages, 3 chunks
Upload	Image-only PDF	failed, "No readable text found, even with OCR."
Upload	Renamed text file; non-PDF extension; 51 MB file	400; 400; 413, nothing written to disk
Upload	Server restarted while a document was processing	Marked failed: "Processing was interrupted"
Chat	Question spanning two pages	Streamed answer citing page 1 and page 2 correctly

Area	Scenario	Result
Chat	Follow-up "how many engineers are on their team?"	Resolved "their" from history; cited page 2
Chat	Notebook with no ready sources	Fixed guidance reply, model not called
Chat	Rate limit mid-answer	Partial text kept, "Retry in 3s" countdown, then Retry enabled; no duplicate question stored
Chat	Stream ends without completion	"Response interrupted before it finished" with Retry
Citations	Answer containing `[[C9]]` (invalid) and `【C2】`	`[[C9]]` dropped; `【C2】` shown as chip 2
Viewer	Click a citation	Opens the cited page; passage highlighted
Persistence	Reload; restart all containers	Notebooks, 4 messages and PDF files intact
Interface	Dark theme; 390 px phone width	Correct rendering; tabbed layout
Formatting	Answer with a table, breaks in cells, headings, a code block, citations inside cells and a raw <div onclick>	Rendered table and code block; chips in cells open the cited page; raw HTML dropped and never run
Search	16 near-identical course codes; question names one code	Top result correct for 16/16 codes (vector search alone: 15/16)
Search	Question made only of stop words ("what is this?")	Vector results returned; no error
OCR	Two scanned pages; a mixed PDF (page 1 text, page 2 scanned); a scanned blank page	Ready with 2 OCR'd pages; ready with 1 OCR'd page and correct page numbers; failed "No readable text found, even with OCR."
OCR	Chat question answered from scanned pages	Cited the OCR'd pages correctly
Study	10-question topic quiz and 20 flashcards on an 8-page lecture	Generated in 2-7 s; flashcards cited all 8 pages; quiz answer keys checked by hand

Area	Scenario	Result
Study	Model output with duplicate options, out-of-range answer, invented citation, broken JSON	Bad items dropped; under half valid gives 502; broken JSON retried once
Study	Second user opens, scores, deletes or generates in the first user's notebook	404 in every case
Study	Browser: generate, answer by keyboard, open citation, score saved, flip and shuffle cards, delete, mocked rate limit, phone width	All passed; removed-source citation shows a message instead of failing
Docker	Fresh stack: migrations 0001 and 0002, scanned PDF, quiz through nginx	OCR'd 2 pages offline; quiz cited the scanned pages

13.3 Defects found and fixed during development

Defect	Cause	Fix
Answers had no sources	gpt-oss writes `【C1】` instead of `[[C1]]`	Normalisation on server and client; stricter prompt
Original model unavailable	`llama-3.3-70b-versatile` returned 404 for the account	Switched to `openai/gpt-oss-120b`; raised the token budget for its reasoning
Mixed-case usernames rejected	Pattern validated before lowercasing	Normalise first, then validate
"Uploading…" stuck after multi-file drop	Mutation callbacks fire only for the latest call	Per-file promises
PDF viewer crashed the page in Docker	nginx served the `.mjs` worker as `application/octet-stream`	MIME rule in nginx; error boundaries around the viewer and routes
Vector search could return too few results	IVFFlat index built on an empty table	HNSW index
Tables in answers shown as raw pipes and tags	The first answer renderer handled only paragraphs, lists, bold and italic	Replaced with react-markdown and remark-gfm; markers rewritten as chip links

Defect	Cause	Fix
Quiz answer key wrong on a "which is NOT" question	Low reasoning effort and negative question wording	Medium reasoning for quizzes; prompt forbids negative questions and same-meaning options
Explanations said "C4 states that…"	Model copied internal excerpt labels into student-facing text	Prompt rule plus server-side rewrite of leaked labels to "the source"
Correct quiz answers had no green tint	An arbitrary colour class was never generated by Tailwind	Dedicated ok-soft colour token

14. Limitations and future work

The current system is built for a single machine and PDF text; the main limits are retrieval quality on large collections, scanned documents, and scaling beyond one backend process.

14.1 Known limitations

Limitation	Effect	Mitigation today
OCR'd pages have no text layer	Citations to scanned pages open the page without highlighting the passage	The hover card still shows the cited text
Character-based chunking	Chunk boundaries ignore sentences and the model's tokeniser	150-character overlap keeps split sentences whole in one chunk
Fixed top-6 retrieval, no reranking	Long answers may need more context than six chunks	Hybrid search finds exact terms; follow-ups can narrow the question
One backend process	In-memory login limiter and local file storage do not scale across servers	Adequate for single-machine deployment
Stateless sessions	Signing out clears the cookie but does not revoke a copied token before its 7-day expiry	`httpOnly` cookie limits token theft

Limitation	Effect	Mitigation today
Shared free LLM quota	Concurrent users exhaust Groq's free rate limit sooner	Clear countdown and retry in the interface
No password reset	A forgotten password cannot be recovered	—
Model availability	Free-tier models change; one used during development was withdrawn	Model is a single setting (`GROQ_MODEL`)
AI-generated quiz questions	A question can occasionally have a wrong or ambiguous answer	Explanation and source page shown for every answer, so it can be checked

14.2 Future work

1. **Reranking.** Rerank the hybrid-search candidates with a cross-encoder to improve precision.
2. **Highlighting on scanned pages**, by storing OCR word positions so the viewer can mark the cited passage.
3. **More source types**: web pages, Word documents and video transcripts can reuse the chunking, embedding and citation pipeline once converted to page- or section-tagged text.
4. **Retrieval evaluation.** A question set with known answer pages to measure recall and citation accuracy when tuning chunk size and top-k.
5. **Scaling.** A job queue for ingestion, object storage for PDFs, and a shared store for rate limiting and token revocation.
6. **Collaboration.** Sharing notebooks between users and multiple chat threads per notebook.
7. **Study features** in the spirit of NotebookLM: generated summaries and study guides, written-answer questions, and spaced-repetition scheduling for flashcards.

15. Conclusion

Folio meets the objectives set in Section 1: users can question their own PDFs, receive answers grounded in those documents, and verify each claim on the highlighted source page, with accounts keeping every user's data private. The design shows that a complete RAG product can run at no cost by keeping embedding and search local and calling a hosted model only to write the answer.

The main lesson is that the reliability of a RAG system rests on the plumbing around the model. Citation markers had to be normalised because the model ignored the requested format; highlighting needed whitespace-insensitive matching because two PDF parsers disagree; and deployment needed explicit streaming and MIME configuration. Each of these was found by running the complete system, which is why end-to-end verification was the core of the testing strategy.

Appendix A. Configuration reference

Variable	Default	Purpose
`GROQ_API_KEY`	— (required)	Groq key for answer generation
`JWT_SECRET`	— (required)	Signs session cookies; startup fails if unset or `change-me`
`GROQ_MODEL`	`openai/gpt-oss-120b`	Chat model on Groq
`POSTGRES_PASSWORD`	`ragpass`	Database password used by Docker Compose
`DATABASE_URL`	`postgresql+asyncpg://raguser:ragpass@localhost:5432/notebooklm`	Database URL outside Docker
`UPLOAD_DIR`	`./uploads`	Where PDFs are stored
`MAX_UPLOAD_MB`	`50`	Largest accepted PDF
`COOKIE_SECURE`	`false`	Set `true` when served over HTTPS
`JWT_EXPIRE_DAYS`	`7`	Session lifetime

Variable	Default	Purpose
`EMBEDDING_MODEL`	`BAAI/bge-small-en-v1.5`	Local embedding model
`EMBEDDING_DIM`	`384`	Vector size; must match the model and the migration
`CHUNK_SIZE`	`800`	Characters per chunk
`CHUNK_OVERLAP`	`150`	Characters shared by neighbouring chunks
`TOP_K`	`6`	Chunks retrieved per question
`HISTORY_MESSAGES`	`6`	Prior messages sent for follow-up context

Appendix B. Repository layout

Backend paths elsewhere in this report (such as `routers/chat.py`) are relative to `backend/app/`.

```
backend/                  FastAPI backend (Python, managed with uv)
  app/
    main.py               app setup, startup recovery, model warm-up
    config.py             settings from the root .env
    database.py           async SQLAlchemy engine and sessions
    models.py             User, Notebook, Document, Chunk, Message
    schemas.py            request and response models
    security.py           bcrypt, JWT, login rate limiter
    deps.py               current user and ownership checks
    routers/              auth, notebooks, documents, chat
    services/
      ingest.py           background PDF processing
```

```
      pdf_parser.py       PDF to per-page text
      chunking.py         overlapping page-tagged chunks
      embeddings.py       local embedding model
      retrieval.py        pgvector similarity search
      llm.py              prompt, Groq streaming, citation handling
      storage.py          PDF files on disk
  alembic/                database migrations
  docker/entrypoint.sh    migrations, then server start
  Dockerfile              backend image
  pyproject.toml, uv.lock Python dependencies
frontend/                 React + TypeScript app
  src/api/                HTTP client, SSE reader, query hooks
  src/lib/                citation logic, formatting
  src/pages/              sign-in, notebooks, workspace
  src/components/         workspace panels (markdown answers, PDF viewer) and
UI parts
  nginx.conf              static serving and /api proxy
  Dockerfile              build and serve image
docker-compose.yml        db, backend, frontend
.env.example              configuration template (copied to .env at the root)
```